Digital Twin Concept & Architecture Guide

How & Where the Graph Digital Twin is Implemented in Our Paper and Codebase

Anshika Keshari
Kalinga Institute of Industrial Technology (KIIT), Bhubaneswar
Research Intern, IIIT Naya Raipur

August 8, 2026

Contents

1 What is a "Digital Twin"? (The Master Analogy) **2**

2 The Digital Twin Concept in Our Smart Grid Project **2**

3 Where is Digital Twin Implemented in Our RESEARCH PAPER? **2**

4 Where is Digital Twin Implemented in Our CODEBASE? **3**

5 Why Our Graph Digital Twin Outperforms Traditional Security **3**

What is a "Digital Twin"? (The Master Analogy)

Real-World Analogy: Formula 1 Race Car & Pit-Lane Simulator
Imagine a **Formula 1 race car** racing on a track:

- **The Physical Car**: Has 300+ digital sensors measuring engine temperature, tire pressure, fuel flow, and speed.
- **The Pit-Lane Computer (Digital Twin)**: In the pit lane, engineers monitor a **live virtual 3D computer model** of the race car. As the real car drives, sensor data streams wirelessly into the virtual model, causing the computer twin to mimic the physical car's exact state in real time.
- **The Cyber Attack Scenario**: A rogue competitor hacks the physical car's tire pressure sensor, injecting fake readings to make the dashboard report a blowout when the tire is actually healthy.
- **How the Digital Twin Saves the Day**:
  1. **Detection & Localization**: The Digital Twin compares incoming tire sensor numbers against physical aerodynamic and suspension laws. It immediately flags an anomaly and pinpoints Tire 3 as the source.
  2. **Explainability**: The engineer's screen displays a red heatmap highlighting Sensor 3's pressure angle input.
  3. **Closed-Loop Healing**: The Digital Twin automatically mutes the hacked sensor signal and substitutes real-time suspension physics estimates, allowing the driver to keep racing safely.

The Digital Twin Concept in Our Smart Grid Project

In our project, we apply this exact concept to **Electrical Power Grids**:

1. **Physical Asset**: The physical IEEE 14-bus or IEEE 30-bus transmission grid streaming SCADA telemetry (V, θ, P, Q).
2. **Virtual Mirror**: A spatial admittance graph $G = (V, E, X, E_{\text{attr}})$ where buses are nodes, transmission lines are edges, and nodal admittance $Y_{\text{bus}} = G_{\text{bus}} + jB_{\text{bus}}$ forms the graph topology.
3. **Bidirectional Communication**:
   - **Physical to Virtual**: SCADA RTUs stream measurements (V, θ, P, Q) into the GDT backend.
   - **Virtual to Physical**: When an attack is detected, the GDT sends quarantine commands back to the control center, muting bad RTU channels and restoring clean state tracking.

Where is Digital Twin Implemented in Our RESEARCH PAPER?

Every major section of our IEEE paper (`docs/latex/Smart_Grid_GDT_IEEE_Paper.tex`) directly maps to a component of the Digital Twin:

Paper Section	Section Title	How Digital Twin Concept is Featured
Section I	**Introduction**	Defines the need for a software-based Graph Digital Twin mirroring grid physics.
Section IV-A	**Digital Twin Coupling**	Formulates bidirectional link between physical SCADA and virtual graph mirror.
Section IV-B	**Virtual Graph Construction**	Converts physical bus signals into node features X and branch admittance into edge attributes.
Section IV-C	**GraphSAGE Backbone**	Details GNN brain inside the Digital Twin running detection and localization at 0.908 ms.
Section IV-D	**Operator Explainability**	Details the Digital Twin's forensic XAI dashboard calculating exact attribution scores.
Section VIII	**Closed-Loop Mitigation**	Details self-healing loop muting bad RTUs and substituting forecasts (99.97% recovery).

Where is Digital Twin Implemented in Our CODEBASE?

The Digital Twin framework is built across clean python modules in `src/` and benchmark scripts in `experiments/`:

- `src/data_generation.py`: Acts as the **Physical Grid Asset**, simulating AC power flow physics using `pandapower`.
- `src/model.py`: Acts as the **Virtual Twin Inference Engine**, constructing PyTorch Geometric graph data objects and running the GraphSAGE multi-task model.
- `src/explainability.py`: Acts as the **Digital Twin Diagnostic Engine**, computing exact Integrated Gradients attributions (θ, V, P, Q).
- `src/cyber_mitigation_demo.py`: Acts as the **Digital Twin Self-Healing Actuator**, quarantining corrupted RTU feeds and substituting load baseline forecasts.
- `src/visualize.py`: Acts as the **Digital Twin Operator UI**, rendering the 3D-styled grid map and attribution heatmap.
- `experiments/evaluate_wls.py`: Benchmarks the Digital Twin against classical Weighted Least Squares (WLS) state estimation.

Why Our Graph Digital Twin Outperforms Traditional Security

Feature / Capability	Traditional SCADA Security	Our Graph Digital Twin (GDT)
Grid Representation	Flat list of numbers	**Spatial Admittance Graph** $G(V, E, Y_{\text{bus}})$
Handling Line Outages $(N-1)$	**Crashes / Fails**	**Zero-Shot Generalization** (92.80% Det F1)
Stealthy FDIA Detection	**Fooled** $(r \leq \tau)$	**Detects 98.25% of Attacks** (0.908 ms latency)
Substation Localization	No node-level pinpointing	**97.62% Node Localization F1**
Operator Explainability	Black-box alarms	**Integrated Gradients Sensor Heatmaps**
Post-Attack Recovery	Open-loop passive warning	**Closed-Loop Self-Healing** (99.97% **MSE recovery**)